

P3300R0

C++ Asynchronous Parallel Algorithms

Published Proposal, 2024-02-15

Author:

[Bryce Adelstein Lelbach \(he/him/his\)](#) (NVIDIA)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Audience:

WG21

Table of Contents

- 1 Introduction**
- 2 Terminology**
- 3 Design**
 - 3.1 Compositional Utilities
- 4 Examples**
 - 4.1 Basics
 - 4.2 Normalize
 - 4.3 Minimum Difference
 - 4.4 Rain Water
 - 4.5 Upper Triangular Cholesky Factorization
- 5 Alternatives**
 - 5.1 Asynchronous Parameters and Logical Return Types
 - 5.2 Attach Execution Policies to Schedulers or Vice Versa
 - 5.3 Require Explicit Execution Policy and/or Scheduler Parameters
- 6 Issues**

6.1 Dynamic Asynchrony

6.2 Lifetimes

Index

Terms defined by this specification

References

Informative References

§ 1. Introduction

This paper outlines a unified design for asynchronous parallel algorithms and scheduler-aware synchronous parallel algorithms for Standard C++.

In C++17, we introduced parallel versions of the standard algorithms, which take an execution policy that describes what form of parallelism is allowed, if any. These execution policy parallel algorithms are powerful, but have two major limitations:

- They are synchronous; they launch the parallel work, and then block the calling execution agent until the work completes.
- There is no way to specify where parallel work is executed.
- There is no way to pass tuning knobs and other parameters for execution.

In C++26, we plan to introduce [senders and schedulers](#), C++'s framework for asynchrony and controlling execution. With senders and schedulers, we can address both of the shortcomings of the execution policy parallel algorithms. We can add new asynchronous parallel algorithms that can be composed together and run on any scheduler. We can also add new versions of the synchronous parallel algorithms whose execution can be controlled by schedulers.

§ 2. Terminology

serial algorithm

A version of an algorithm that does not take or return an execution policy, sender, or scheduler. E.g. All of the standard algorithms prior to C++17.

parallel algorithm

A version of an algorithm that either returns a sender or takes an execution policy, sender, or scheduler. It does not necessarily execute in parallel.

synchronous parallel algorithm

A parallel algorithm that blocks the calling execution agent until the parallel work that it has launched completes.

synchronous unscheduled parallel algorithm

A synchronous parallel algorithm that takes an execution policy and no sender or scheduler. Therefore, users cannot control where it executes. E.g. the C++17 parallel algorithms.

synchronous scheduled parallel algorithm

A synchronous parallel algorithm that takes an execution policy and a sender or scheduler. Therefore, users can control where it executes.

asynchronous parallel algorithm

A parallel algorithm that returns a sender and does not block pending the completion of the parallel work it launched.

predecessor sender

The single sender parameter of a synchronous scheduled parallel algorithm or an asynchronous parallel algorithm.

§ 3. Design

We will introduce two new types of parallel algorithms: synchronous scheduled and asynchronous. Both will take a single sender parameter and no scheduler or execution policy parameter. This sender represents prior work that must complete before the algorithm is evaluated.

```
algorithm(a, b, c, d); // Serial
algorithm(pol, a, b, c, d); // Synchronous unscheduled parallel (C++17)
T t = algorithm(snd, a, b, c, d); // Synchronous scheduled parallel (New)
sender_of<T> auto s = async::algorithm(snd, a, b, c, d); // Asynchronous (New)
```

Senders have queryable attributes. A parallel algorithm that takes a sender determines its scheduler and execution policy from the sender by querying the appropriate attribute. A scheduler may be attached to a sender with `snd = schedule(sch)` or `snd | transfer(sch)`. An execution policy may be attached to a sender with `snd | attach_execution_policy(pol)`. Both may be attached at the same time with `snd = schedule(sch, pol)`, `snd | transfer(sch, pol)`, or `on(sch, pol, snd)`.

If the execution policy attached cannot be satisfied by the scheduler, a compilation error occurs. If no execution policy is attached and the scheduler has no default, a compilation error occurs. If the sender passed to a synchronous scheduled parallel algorithm has no scheduler attached to it, a compilation error occurs.

Synchronous scheduler parallel algorithms take a `sender_of<void>`. They are sender consumers - they will connect their predecessor sender, block pending its completion, and then return a non-sender value. They must use `sync_wait` to block pending the completion of both their predecessor sender and the work they create, to ensure that they block in a manner that can be customized by scheduler and sender authors.

Asynchronous parallel algorithms are sender adaptors - they take a sender and return a sender. The returned sender will send the result you would get by calling the serial algorithm (if it's non-void), followed by the values sent by the predecessor sender.

```
sender_of<T> auto s0 = async::algorithm(schedule(sch), a, b, c, d);

sender_of<int, bool> auto s1 = just(17, true);
sender_of<T, int, bool> auto s2 = async::algorithm(s1, a, b, c, d);
```

Asynchronous parallel algorithms are pipeable.

```
sender_of<U> auto s0 = schedule(sch)
    | async::algorithm_x(a, b, c, d) // Result is type `T`.
    | then([] (T t) -> void { /* ... */ })
    | async::algorithm_y(b, e, f); // Result is type `U`.

sender_of<U, V> auto s1 = just(17, true);
    | async::algorithm_x(a, b, c, d)
    | then([] (T t, int i, bool b) -> V { /* ... */ })
    | async::algorithm_y(b, e, f);
```

The senders returned by asynchronous parallel algorithms are awaitable, so we can easily use them in coroutines.

```
R serial {
    return f(algorithm_x(a, b, c, d));
}

sender_of<R> auto pipes() {
    return just() | async::algorithm_x(a, b, c, d) | then(f);
}

sender_of<R> auto coroutines() {
    co_return f(co_await async::algorithm_x(just(), a, b, c, d));
}
```

The asynchronous parallel algorithms will be based on the more modern `std::ranges` algorithms. Because they differ in return type and in semantics, all asynchronous parallel algorithm overloads will live in a new `std::async` namespace. The synchronous scheduled parallel algorithms will provide both legacy iterator-based overloads in `std` (next to their existing counterparts that take execution policies) and new `std::ranges` overloads. For completeness, execution policy overloads of `std::ranges` algorithms should be added as well. A separate paper on the related but orthogonal topic of parallelizing `std::ranges` algorithms and range adaptors is forthcoming.

There is precedent for this overall design in `bulk`. `bulk` is a sender adaptor, does not take an explicit scheduler or execution policy, and passes through the values sent by its predecessor.

§ 3.1. Compositional Utilities

`select` and `with` are sender adaptors that aid composition of asynchronous parallel algorithms. `select<N, M, ...> (snd)` chooses which values from a sender should be sent and in what order. `with(snd, f)` invokes `f` with the values sent by `snd`, and then sends those same values, along with the result of the invocation.

```
template <std::size_t... S>
sender auto
select(sender_of<Args> auto args, F f);
// (args), (S[0], S[1], ...), f -> f(args[S[0]], args[S[1]], args[...], ...)
```

```

sender_of<A, B, ..., invoke_result_t<F, A, B, ...>> auto
with(sender_of<A, B, ...> auto, F f);
// (a, b, ...), f -> (a, b, ..., F(a, b, ...))

```

`send_values(t...)` is a function that returns an unspecified tuple-like type that indicates to `then` and similar adaptors that `t...` should be sent as values by the sender returned from the sender adaptor. Likewise, `send_error(e)` indicates to `then` and similar adaptors that `e` should be sent as an error by the returned sender.

§ 4. Examples

§ 4.1. Basics

```

auto fgh = just() | for_each(v, f) | for_each(v, g) | for_each(v, h);

// Range view chaining.
auto rolling_max = rng | slide(N) | transform(max_element);

// Asynchronous parallel algorithm chaining.
auto unique_sort = just() | async::sort(v) | async::unique(v);

```

§ 4.2. Normalize

```

auto normalize_serial(range auto&& v) {
    auto mx = fold_left(v, ranges::max{});
    return transform(v, views::repeat(mx), begin(v), divides);
}

auto normalize_parallel(range auto&& v) {
    auto mx = fold_left(par, v, ranges::max{});
    return transform(par, v, views::repeat(mx), begin(v), divides);
}

auto normalize_async(range auto&& v) {
    return async::fold_left(v, ranges::max{})
        | let_value([] (auto mx, range auto&& v) {
            return async::transform(
                just(), v, views::repeat(mx), begin(v), divides);
        });
}

auto normalize_coroutine(range auto&& v) {
    auto mx = async::fold_left(just(), v, ranges::max{}) | split;
    co_return async::transform(mx, v, views::repeat(co_await mx), begin(v), divides);
}

```

§ 4.3. Minimum Difference

```
auto minimum_difference_serial(range auto&& v) {
    vector<T> tmp(size(v));
    sort(v);
    adjacent_difference(v, begin(tmp));
    return *min_element(tmp | drop(1));
}

sender auto minimum_difference_async(range auto&& v) {
    return just(vector<T>(size(v)))
        | let_value([&v] (range auto&& tmp) {
            return async::sort(just(), v)
                | async::adjacent_difference(v, begin(tmp))
                | async::min_element(tmp | drop(1));
        });
}

sender auto minimum_difference_coroutine(range auto&& v) {
    vector<T> tmp(size(v));
    auto s0 = async::sort(just(), v);
    auto s1 = async::adjacent_difference(s0, v, begin(tmp));
    co_return *co_await async::min_element(s1, tmp | drop(1));
}
```

§ 4.4. Rain Water

```
int rain_water_serial(range auto&& v) {
    vector<T> tmp(size(v), 0);
    auto it = max_element(v);
    inclusive_scan(begin(v), next(it), begin(tmp), ranges::max{});
    inclusive_scan(rbegin(v), reverse_iterator(it), rbegin(tmp), ranges::max{});
    return transform_reduce(tmp, cbegin(v), 0, plus<>{}, minus<>{});
}

sender_of<int> auto rain_water_async(range auto&& v) {
    return just(vector<T>(size(v)))
        | let_value([&v] (range auto&& tmp) {
            auto sit = async::max_element(just(), v) | split;

            auto sleft = sit
                | let_value([&v] (auto it) {
                    return async::inclusive_scan(
                        just(), begin(v), next(it), begin(tmp),
                        ranges::max{});
                });

            auto sright = sit
                | let_value([&v] (auto it) {
                    return async::inclusive_scan(
```

```

        just(), rbegin(v), reverse_iterator(it), rbegin(tmp),
        ranges::max{}));
    });

    return when_all(sleft, sright)
        | async::transform_reduce(tmp, cbegin(v), 0, plus<>(), minus<>());
});
}

sender_of<int> auto rain_water_coroutine(range auto&& v) {
    vector<T> tmp(size(v), 0);
    auto sit = async::max_element(just(), v) | split;
    auto sleft = async::inclusive_scan(
        sit, begin(v), next(co_await sit), begin(tmp), ranges::max{});
    auto sright = async::inclusive_scan(
        sit, rbegin(v), reverse_iterator(co_await sit), rbegin(tmp), ranges::max{});
    co_return transform_reduce(tmp, cbegin(v), 0, plus<>{}, minus<>{});
}

```

§ 4.5. Upper Triangular Cholesky Factorization

```

void upper_triangular_cholesky_factorization_parallel(
    la_matrix A, la_vector b, la_vector x)
{
    // Solve  $U^T c = b$ , using x to store c
    triangular_matrix_vector_solve(
        par,
        transposed(A), upper_triangle, explicit_diagonal,
        b,
        x,
        divides);

    // Solve  $U x = c$ , overwriting x with result
    triangular_matrix_vector_solve(
        par,
        A, upper_triangle, explicit_diagonal,
        x,
        divides);
}

sender_of<la_vector> upper_triangular_cholesky_factorization_async(
    la_matrix A, la_vector b, la_vector x)
{
    return just(x)
        | async::triangular_matrix_vector_solve(
            transposed(A), upper_triangle, explicit_diagonal,
            b,
            x,

```

```

        divides);
    // Solve  $Ux = c$ , overwriting  $x$  with result
    | async::triangular_matrix_vector_solve(
        A, upper_triangle, explicit_diagonal,
        x,
        divides);
}

```

§ 5. Alternatives

§ 5.1. Asynchronous Parameters and Logical Return Types

We explored a design where the parameters to the algorithms (ranges, iterators, invocables) could be provided asynchronously by the predecessor sender instead of being supplied at the time the asynchronous algorithm is invoked. If the predecessor sender sends N values, they correspond to the first N parameters of the parallel algorithm. None of the parameters may be skipped; they bind from the front, like `bind_front`. Any parameters that are not sent by the sender must be passed after the predecessor parameter. The sender may send no parameters, in which case they all must be provided after the predecessor sender parameter.

```

// Synchronous scheduled parallel (New)
value = algorithm(schedule(sch), a, b, c, d);
value = algorithm(transfer_just(sch, a), b, c, d);
value = algorithm(transfer_just(sch, a, b), b, c, d);
value = algorithm(transfer_just(sch, a, b, c), d);
value = algorithm(transfer_just(sch, a, b, c));

// Asynchronous parallel (New)
snd = async::algorithm(just(), a, b, c, d);
snd = async::algorithm(just(a), b, c, d);
snd = async::algorithm(just(a, b), b, c, d);
snd = async::algorithm(just(a, b, c), d);
snd = async::algorithm(just(a, b, c, d));

```

The goal of this model was composability. We wanted to enable the same point-free style programming used by range adaptor pipelines, where the inputs and outputs of each operation are not explicitly named and flow into each other.

We found that this model worked best when you are working with a single range and performing operations in-place:

```

auto fgh = just(v) | for_each(f) | for_each(g) | for_each(h);

auto unique_sort = just(v) | async::sort | async::unique;

```

However, we found that the model broke down when writing more complex code. We often had to use the `with` and `select` utilities and additionally insert `then` adaptors that would adjust parameters in between algorithm invocations - increment an iterator by one, dereference an iterator, turn a scalar into a range view, get an iterator from a range, etc.

```

auto normalize_async(range auto&& rng) {
    return async::fold_left(rng, max)
        | then([] (auto rng, auto mx) {

```



```

        return send_values(rng, repeat(mx));
    })
    | inplace(async::transform(divides));
}

sender_of<la_vector> upper_triangular_cholesky_factorization_async(
    la_matrix A, la_vector b, la_vector x)
{
    return just(transposed(A), upper_triangle, explicit_diagonal, b, x)
        | with(async::triangular_matrix_vector_solve(divides))
        // Receives (transposed(A), upper_triangle, explicit_diagonal, b, x)
        // Sends (transposed(A), upper_triangle, explicit_diagonal, b, x)
        | select<0, 1, 2, 4>(then([] (auto A, auto tri, auto dia, auto x) {
            return send_values(transposed(A), tri, dia, x);
        })) // Drop b and untranspose A.
        // Sends (A, upper_triangle, explicit_diagonal, x)
        | async::triangular_matrix_vector_solve(divides);
}

```

We were particularly plagued by mismatches between range parameters (which are only used for the primary input to algorithms), iterator parameters (which are used for auxiliary inputs and outputs to algorithms), and non-scalar return types (most algorithms return iterators-past-the-end, which can't be fed in as the next primary input, which needs to be a range).

Many algorithms are not in-place, such as `transform`. An `inplace(snd, adaptor)` that transforms an out-of-place asynchronous algorithm into an in-place one helps with that:

```

sender_of<decltype(Adaptor{}(R{}, iterator_t<R{}>))> auto
inplace(sender_of<R> r, Adaptor a);
// r, a -> a(r, begin(r))

auto fgh = just(v)
    | inplace(transform(f))
    | inplace(transform(g))
    | inplace(transform(h));

```

As part of this design, we planned to have the asynchronous parallel algorithms return a sender that sends the logical output of the algorithm, suitable for piping into another parallel algorithm. This asynchronous value will often differ from the return value of the corresponding non-parallel algorithm. For example, `transform` returns a single iterator to the end of the transformed range, but `async::transform` should send the transformed range. This is implementable because parallel algorithms require forward iterators.

Algorithm	Non-Parallel Returns...	Asynchronous Sends...
<code>transform(Range0 rng0, [Range1 rng1,] Iterator out, F f)</code>	Iterators to the last transformed element of <code>rng0</code> and <code>rng1</code> and an iterator to the element past the last transformed element of <code>out</code> .	The range of transformed elements of <code>out</code> , e.g. <code>subrange(out, advance(out, N))</code> , where <code>N</code> is the number of elements transformed.
<code>for_each(Range rng, F f)</code>	An iterator to the element past the last transformed element of <code>rng</code> and the object <code>f</code> .	<code>rng</code> .
<code>reduce(Range rng, T init, F f)</code>	The sum as a <code>T</code> .	The sum as a <code>T</code> .
<code>fold_*(Range rng, [T init,] F f)</code>	The sum as a <code>T</code> .	The sum as a <code>T</code> .
<code>find_if(Range rng, Pred p)</code>	An iterator to the found element.	An iterator to the found element.
<code>(min max)_element(Range rng, Comp c)</code>	An iterator to the found element.	An iterator to the found element.
<code>copy(Range from, Iterator to)</code>	An iterator to the last copied element of <code>from</code> and an iterator to the element past the last copied element of <code>to</code> .	The range of copied elements of <code>to</code> , e.g. <code>subrange(to, advance(to, N))</code> , where <code>N</code> is the number of elements copied.
<code>copy_if(Range from, Iterator to)</code>	An iterator to the last copied element of <code>from</code> and an iterator to the element past the last copied element of <code>to</code> .	The range of copied elements of <code>to</code> , e.g. <code>subrange(to, advance(to, M))</code> , where <code>M</code> is the number of elements copied.
<code>*_scan(Range rng, Iterator out, T init, F f)</code>	An iterator to the element past the last scanned element of <code>out</code> (this is what the non-parallel overload returns, as there's no parallel overload yet)	The range of scanned elements of <code>out</code> , e.g. <code>subrange(out, advance(out, N))</code> , where <code>N</code> is the number of elements scanned.
<code>sort(Range rng, Comp c)</code>	An iterator to the last sorted element.	<code>rng</code> .
<code>unique(Range rng, Comp c)</code>	The range of unique elements of <code>rng</code> , e.g. <code>subrange(begin(rng), advance(begin(rng), N))</code> , where <code>N</code> is the number of unique elements.	The range of unique elements of <code>rng</code> , e.g. <code>subrange(begin(rng), advance(begin(rng), N))</code> , where <code>N</code> is the number of unique elements.

<code>partition(Range rng, Comp c)</code>	The range of the group of <code>rng</code> for which <code>c</code> is false (the second group), e.g. <code>subrange(advance(begin(rng), M), end(rng))</code> , where <code>M</code> is the number of elements for which <code>c</code> is true.	The range of the group of <code>rng</code> for which <code>c</code> is false (the second group), e.g. <code>subrange(advance(begin(rng), M), end(rng))</code> , where <code>M</code> is the number of elements for which <code>c</code> is true.
---	---	---

We eventually reached the conclusion that this whole approach was too complex, and that it would be better to use `let_value` and named variables to handle asynchronous parameters. The major downside to this approach is that `let_value` requires dynamic parallelism, which is undesirable on some platforms. This is discussed more in [§ 6.1 Dynamic Asynchrony](#).

§ 5.2. Attach Execution Policies to Schedulers or Vice Versa

[\[P2500R2\]](#) suggests a different approach for synchronous scheduled parallel algorithms. It proposes that we attach execution policies to schedulers, and then pass these policy-aware schedulers as a parameter to the synchronous scheduled parallel algorithms, instead of passing a sender parameter as we suggest.

We believe that attaching execution policies to schedulers or vice versa is not a clean design. We already have a property-like mechanism for attaching things to senders/receivers (`get_env`, sender attributes, and receiver environments). If we also allow execution policies to have things attached to them, we'll end up introducing an additional system which may interact poorly with the sender/receiver property mechanism.

Many of the sender/receiver properties are applicable to the synchronous scheduled parallel algorithms too. How do we pass an allocator to a synchronous parallel algorithm? How do we pass a stop token to a synchronous scheduled parallel algorithm? Do we attach those to schedulers as well? We'll end up needing every sender/receiver property on schedulers as well.

If both senders/receivers and schedulers can have the same kinds of things attached to them, and schedulers can be attached to senders/receivers, we can end up with collisions and confusion. Should an asynchronous parallel algorithm use the execution policy attached to the sender/receiver? Or should it use the execution policy attached to the scheduler that's attached to the sender/receiver? What if both exist and disagree?

We have ample experience with property systems on execution policies in the Thrust library, which was the inspiration for C++'s parallel algorithms and has over 15 years of field experience and hundreds of thousands of users. Initially, we introduced one property for execution policies, but over time, user demand for tuning knobs and controls led to the addition of others. Today, Thrust's execution policies can have the moral equivalent of schedulers (streams), allocators, and asynchronous dependencies attached to them. The interactions between these different properties and their lifetime model is brittle and difficult to understand. Our field experience with this model has largely been negative.

It is much simpler for us to have one property system (the existing one for senders/receivers), attach both schedulers and execution policies via that property system, and then parameterize both the synchronous and asynchronous parallel algorithms on senders, not execution policies.

§ 5.3. Require Explicit Execution Policy and/or Scheduler Parameters

Another alternative design would be to require explicit execution policy and/or scheduler parameters on every parallel algorithm invocation. This might be reasonable, albeit verbose, for the synchronous scheduled parallel algorithms. For the asynchronous parallel algorithms, this would significantly constrain their functionality and make it harder to write generic code.

You may not know which execution policy or scheduler should be used at the time that you are calling an asynchronous parallel algorithm; that's fine, because they can be attached later. If you do know that a particular execution policy or scheduler is needed, that's also fine, because you can attach them early.

```
// Early attaching: We know the types and operations and can decide what to use.
auto unique_sort(vector<int>& v) {
    return schedule(system_scheduler, par) | sort(v) | unique(v);
}

// Late attaching: We don't know all the types and operations.
auto pattern(range auto&& v, auto f, auto g, auto h) {
    return just() | async::transform(v, f) | async::then(g) | async::transform(v, h);
}

// The decision about execution policy and scheduler to use should be local to
// the knowledge of the types and operations.
auto s = on(sch, pol, pattern(d, x, y, z));
```

We do not require that users specify a scheduler for every sender adaptor today; we should not require this of asynchronous parallel algorithms either.

§ 6. Issues

§ 6.1. Dynamic Asynchrony

Asynchrony is dynamic when asynchronous work is enqueued by other asynchronous work. In some parallel programming environments, dynamic asynchrony is either expensive or impossible to implement.

This is a common issue for GPU programming models like CUDA - launching new asynchronous work from the GPU is either expensive or impossible. For CUDA in particular, we have a feature called CUDA Dynamic Parallelism (CDP) that allows us to launch new GPU work from the GPU. The original implementation of CDP had such significant overheads that it was almost never used. CUDA recently released a new CDP model that offers much better overheads - but the launch latency is still approximately 2x worse than CPU launch latency.

Most of the senders framework can be implemented without dynamic asynchrony. However, that is not the case for `let_value`, which is inherently dynamic, as we cannot know what the next sender will be until we connect and invoke the user-provided invocable. In our current implementation of CUDA schedulers, `let_value` will block until the entire predecessor chain completes on the CPU side, introducing a substantial latency bubble that destroys performance. We believe a better implementation is possible using the new CUDA CDP model, however this will still have substantial overheads that are best avoided.

For this reason, the NVIDIA mindset on `let_value` has typically been that it is to be avoided or used sparingly, as it will have performance issues on our platform. This is one of the reasons we pursued a point-free-style design, as it was hoped this would limit the need for `let_value`. However, we do not see a path for an elegant point-free-style design, so we are now resigned to using `let_value`, despite its performance pitfalls.

§ 6.2. Lifetimes

While most of the serial algorithms take arguments by value, the range-based ones tend to take ranges by universal reference. That's fine because they complete immediately. But it may be a bit of a footgun for asynchronous parallel algorithms, just as it is for range views.

§ Index

§ Terms defined by this specification

[asynchronous parallel algorithm](#), in § 2

[parallel algorithm](#), in § 2

[predecessor sender](#), in § 2

[serial algorithm](#), in § 2

[synchronous parallel algorithm](#), in § 2

[synchronous scheduled parallel algorithm](#), in § 2

[synchronous unscheduled parallel algorithm](#), in § 2

§ References

§ Informative References

[P2300R7]

Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. [`std::execution`](#). 21 April 2023. URL: <https://wg21.link/p2300r7>

[P2500R2]

Ruslan Arutyunyan, Alexey Kukanov. [C++ parallel algorithms and P2300](#). 15 October 2023. URL: <https://wg21.link/p2500r2>